

INTRODUCCIÓN A LA PROGRAMACIÓN ORIENTADA A OBJETOS Y DESARROLLO DE CLASES

UD3 Y UD4



PRINCIPIOS BÁSICOS DE PROGRAMACIÓN ORIENTADA A OBJETOS

Encapsulación
Herencia
Polimorfismo

Las clases se crean cuando:

- La encapsulación protege los datos evitando que puedan ser modificados o accedidos de manera directa y descontrolada, lo que ayuda a mantener la integridad de los datos y facilita el mantenimiento del código.
- Esto se logra ocultando los detalles internos de la implementación y exponiendo solo una interfaz pública segura para interactuar con el objeto.
- Este mecanismo es fundamental para garantizar la seguridad, la modularidad y la claridad en el diseño de programas orientados a objetos.
- Como se verá a continuación, para aplicar la encapsulación, se va a hacer uso de lo que llamaremos métodos.

CARACTERÍSTICAS DE LOS OBJETOS

variable $\approx$ objeto
tipo $\approx$ clase

Por ejemplo:

`int x;` // declaro la variable 'x' del tipo entero

`Cliente c;` // declaro el objeto 'c' de la clase Cliente

LAS CLASES NOS PERMITEN DEFINIR NUESTROS PROPIOS TIPOS, MÁS ELABORADOS Y CON MÁS FUNCIONALIDADES

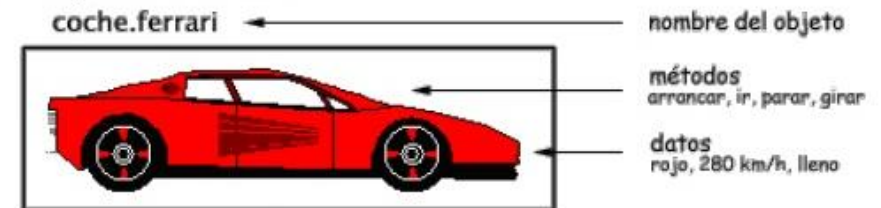
Las clases se crean cuando:

- Hay un concepto nuevo de un problema a solucionar.
- Cuando hay un tipo de datos nuevo.
- A veces se puede crear una clase sólo para contener el método "main" del programa, sin responder a ningún concepto o tipo de datos.

Clase:
Coche



♦ **Objeto:** *Ferrari*



ESTRUCTURA DE UN CLASE

La clase está formada por las características que la definen, también llamadas atributos, y las acciones que puede llevar a cabo, que pasaremos a llamar métodos.

```
tipoAcceso class NombreClase
{
    tipoAcceso TipoAtributo nombreAtributo;
    tipoAcceso TipoDevuelto nombreMétodo (TipoParametro1 nombreParametro1, TipoParametroN nombreParametroN)
    {
        Tipo nombreVariableLocal;
        // otras sentencias;
        /* Si el tipo de retorno es distinto del void deberá tener la palabra reservada return y una variable o valor asociado del mismo
        tipo que el TipoDevuelto */
    }
}
```

ATRIBUTOS

tipoAcceso tipoAtributo nombreAtributo;

Los atributos se pueden inicializar directamente en la declaración o a través de los métodos (**preferiblemente lo segundo**). Los atributos que no se inicializan lo hacen por defecto. Por ejemplo, un entero con **0** y un objeto con **null**.

El tipo de acceso puede ser de varios tipos:

public, **protected** o **private**

Si no se pone nada, se interpreta que es tipo “**paquete**” (se explicará en la siguiente clase).

Si declaramos un atributo con tipo de acceso **public** se puede acceder libremente al atributo cuando se declare un objeto.

Si declaramos un atributo con tipo de acceso **private** sólo se puede acceder a dicho atributo desde dentro de la clase que define dicho atributo.

Si no se pone nada se puede acceder libremente al atributo cuando se declare un objeto, pero siempre desde el mismo paquete (esto se verá más adelante).

El tipo de acceso **protected** se explicará cuando se estudie el tema de la herencia

MÉTODOS

Los métodos determinan el **comportamiento** de los objetos. Se declaran como se vio en la estructura de clase. Los objetos creados de una clase tendrán disponibles los métodos declarados en ella.

Los métodos se pueden clasificar en tres tipos:

- **Métodos de consulta**, que sirven para extraer información (de los atributos) de los objetos. Son los llamados métodos **getters**.
- **Métodos de modificación**, que sirven para modificar uno o varios atributos de los objetos. Son los llamados métodos **setters**.
- **Métodos de comportamiento**, que realizan operaciones relacionadas con los objetos.

MÉTODOS

```

tipoAcceso TipoDevuelto nombreMétodo (TipoParametro1 nombreParametro1, TipoParametroN
nombreParametroN)
{
    Tipo nombreVariableLocal;
    // otras sentencias;
    /* Si el tipo de retorno es distinto del void deberá tener la palabra reservada return y una variable o valor
    asociado del mismo tipo que el TipoDevuelto */
}

```

Se recomienda a los alumnos que utilice los métodos con el tipo de acceso público, a no ser que quieran realizar una acción que sólo sea necesaria dentro de la clase. En este caso cuando se cree un objeto no se podrá hacer la llamada al método que sea creado como acceso privado.

```

public int suma (int sumando1, int sumando2, int sumando3)
{ int valorADevolver; // variable local
valorADevolver = sumando1 + sumando2 + sumando3;
return valorADevolver; }

```

La variable **valorADevolver** tiene que ser del mismo tipo que el tipo que devuelve el método.

MÉTODO constructor, AUTOREFERENCIA this

Para reservar este espacio debemos utilizar el constructor de la clase. El constructor es un método especial que **se llama igual que la clase** en cuestión y que **no tiene tipo de retorno**. Aparte de reservar espacio en memoria, se suele aprovechar el método constructor para dar valores iniciales a los atributos del objeto, y por tanto particularizarlo. Por ejemplo:

```
public Persona(String nombre, int edad, long dni) {  
    this.nombre = nombre;  
    this.edad = edad;  
    this.dni = dni;  
}
```

Si no nos interesa dar valores iniciales, escribiríamos:

```
public Persona() { }
```

Sin embargo, este constructor es tan sencillo que Java no exige que lo escribamos, sino que lo utilizará automáticamente en caso de que no aportemos ningún otro constructor (como el del ejemplo anterior). Con este constructor por defecto se asignarán a los atributos de estos los valores por defecto explicados anteriormente (null, 0, false, etc.). **Es importante destacar que este constructor por defecto que nos ofrece Java sin necesidad de escribirlo no estará disponible en caso de que nosotros escribamos nuestro propio constructor.** Nota: Si dentro del constructor se produce una excepción el objeto no se crea.

Palabra reservada: `this`. Sirve para nombrarse a si mismo.

MÉTODO constructor copia

Un **constructor copia** es un constructor especial que crea un nuevo objeto duplicando el estado (valores de atributos) de otro objeto del mismo tipo que ya existe. Sirve para crear objetos independientes, pero con los mismos datos.

Definición sencilla: Es un constructor que recibe como parámetro un objeto de la misma clase y copia sus atributos para inicializar un nuevo objeto.

En el ejemplo, p2 es un nuevo Punto que copia los valores de p1 mediante el constructor copia.

```
public class Punto
{ private int x, y; // Constructor normal
  public Punto(int x, int y)
  { this.x = x; this.y = y; } // Constructor copia
  public Punto(Punto p) { this.x = p.x; this.y = p.y; }
  public void mostrar() { System.out.println("x: " + x + ", y: " + y); }
  public static void main(String[] args)
  {
    Punto p1 = new Punto(10, 7);
    Punto p2 = new Punto(p1); // Usando el constructor copia
    p1.mostrar(); // Imprime: x: 10, y: 7
    p2.mostrar(); // También imprime: x: 10, y: 7
  }
}
```

SOBRECARGA DE MÉTODOS

Hay sobrecarga cuando existen varios métodos que se llaman igual pero que pueden usar parámetros distintos y/o devuelven resultados distintos.

Ejemplo de sobrecarga de operadores: +

Pregunta: Además de algunos operadores ¿en qué otro sitio habéis visto la sobrecarga de métodos hasta ahora?

MÉTODOS paso por valor, paso por referencia I

- **Paso por valor:** cuando se pasa un parámetro a una función, se le envía una copia del valor original. Entonces, cualquier cambio hecho al parámetro dentro de la función **no afecta** al valor original fuera de la función.
- **Paso por referencia:** cuando se pasa un parámetro a una función, se le envía una referencia (dirección) al valor original. Por tanto, los cambios realizados dentro de la función **afectan** al valor original.

Estos conceptos determinan si la función puede modificar directamente lo que se le pasa o solo manipular una copia sin alterar el original.

En Java las variables de tipo primitivo se pasan por valor, los objetos por referencia, pero dicha referencia se pasa por valor.

MÉTODOS paso por valor, paso por referencia II

```
public class Persona
{
    String nombre;
    int edad;
    Persona(String nombre, int edad)
    {
        this.nombre = nombre;
        this.edad = edad;
    }
}
```

```
public class Ejemplo
{
    public void modificarEdad(int edad)
    { edad = 30; // cambia solo la copia local }
    public void modificarPersona(Persona p) {
        p.nombre = "Juan";
        p.edad = 30; // modifica directamente el objeto original
    }
    public void ejecutarEjemplo() {
        int edad = 20;
        modificarEdad(edad);
        System.out.println("Edad después modificarEdad: " + edad);
        // sigue siendo 20
        Persona persona = new Persona("Ana", 20);
        modificarPersona(persona);
        System.out.println("Persona después modificarPersona: " + persona.nombre + ", "
+ persona.edad); // Juan, 30 }
    public static void main(String[] args)
    { Ejemplo ejemplo = new Ejemplo();
      ejemplo.ejecutarEjemplo(); }
}
```

CREACIÓN DE OBJETOS Y VARIABLE

REFERENCIA A OBJETOS, ALIAS

Para crear objetos es necesario ejecutar esta sentencia, llamando al método constructor:

```
TipoClase nombreObjeto = new TipoClase();
```

Por ejemplo: `Persona` juan = `new` `Persona`("Juan Jiménez", 54, 2535434);

Referencias compartidas: alias

Es posible que dos variables del `TipoClase` referencien (apunten) a un único objeto. Por ejemplo, si declaramos una variable:

```
Persona juan = new Persona();
```

Y a continuación declaramos otra variable y le asignamos el valor de la primera:

```
Persona raul = juan;
```

En este caso no tenemos dos objetos `juan` y `raul`, sino dos referencias al mismo objeto, ya que como se explicó antes, sólo se crea el objeto en memoria cuando se ejecuta el constructor `Persona()`. Por tanto, puede haber muchas referencias distintas que apunten a un solo objeto.

ELEMENTOS DE CLASE (STATIC)

Existen elementos de una clase (entendiendo por elementos tanto métodos como atributos) que están disponibles cuando creamos un objeto de una determinada clase:

miObjeto.atributo

Pero también existen elementos que están disponibles acudiendo directamente a la clase, y no al objeto (que puede no existir), por ejemplo:

el método main o System.out.println("hola");

¿Cómo es posible esto? Porque llevan la palabra reservada **static**, lo que significa que son elementos de clase. Estos elementos son compartidos por todas las instancias de las clases, por tanto, su particularidad es que no existe un elemento por cada objeto, sino un único elemento para toda la clase.

CLASES PREDEFINIDAS: ENVOLTORIO (WRAPPERS)

Java aporta muchas clases predefinidas, que no necesitan ser importadas y que están siempre disponibles para el programador.

Las primeras que vamos a estudiar son las clases “envoltorio” (**wrappers**), que corresponden a cada tipo primitivo y que aportan interesantes métodos de clase, habitualmente relacionados con la conversión de datos a dicho tipo.

Ya vimos algún ejemplo en las primeras clases, como cuando convertíamos una cadena de texto leída por el usuario a un número entero:

```
Integer.parseInt("345");
```

Estas clases se encuentran en el paquete **java.lang**

Véase API (Application Programming Interface):

<https://docs.oracle.com/en/java/javase/21/docs/api/index.html>

OTRAS CLASES PREDEFINIDAS DEL PAQUETE JAVA.LANG

El otro tipo de clases predefinidas no tiene que ver con tipos primitivos, sino que simplemente ofrece funcionalidades que nos serán de interés, por ejemplo, la de elementos matemáticos (Math). Así, en lugar de describir nosotros mismos la constante PI, podemos tomarla de la clase predefinida Math:

```
double perimetro = 2 * Math.PI * radio;
```

AMBITO

Los bloques, delimitados en Java por los símbolos “{” y “}”, definen ámbitos de declaración. Esto significa que en cada bloque tendremos acceso a un determinado conjunto de elementos (ya sean clases, atributos, métodos, variables locales...).

EJERCICIO:

```
1 public class c1
2 {
3     int x, y;
4     public void m1 ()
5     {
6         x = y;
7         this.m2 (y) ;
8     } //m1
9     public void m2 ( int y )
10    {
11        int z;
12        x = y;
13        this.m1 () ;
14    } //m2
15 } //c1
```

PAQUETE, ENCAPSULACIÓN

- Los paquetes son una forma de agrupación de clases, y contribuyen a conseguir la encapsulación, uno de los principios fundamentales de la programación orientada a objetos.
- El **primer nivel** de encapsulación es la clase, que agrupa datos y funciones (atributos y métodos).
- El paquete constituye un **segundo nivel**, permitiendo la agrupación de clases y también de otros paquetes (llamados subpaquetes).
- El uso de paquetes facilita la reutilización de ciertos componentes que ya han sido escritos, y mejora también la organización y estructura de nuestro código.
- Al igual que otros lenguajes, esta organización se realiza mediante librerías o, si hablamos de Java, "packages".
- Los paquetes no son sólo una agrupación conceptual, sino que se corresponden con una agrupación física. En concreto, en la mayoría de los entornos de programación cada paquete se corresponde con un directorio, y las clases incluidas en el paquete se encuentran almacenadas en dicho directorio. De igual forma, un subpaquete se correspondería con un subdirectorio.
- Para indicar que una clase pertenece a un paquete se incluye una primera línea en el archivo de clase indicando "package" y el nombre del paquete: **package paquete1.paquete2;**
- Para importa una clase de un paquete se usa: **import paquete1.paquete2.Clase;**

Véase ejemplo apuntes figuras

TIPO DE ACCESOS

El tipo de acceso puede ser de varios tipos:

public, **protected** o **private**

Si no se pone nada, se interpreta que es tipo “**paquete**” (se explicará en la siguiente clase).

Si declaramos un atributo con tipo de acceso **public** se puede acceder libremente al atributo cuando se declare un objeto.

Si declaramos un atributo con tipo de acceso **private** sólo se puede acceder a dicho atributo desde dentro de la clase que define dicho atributo.

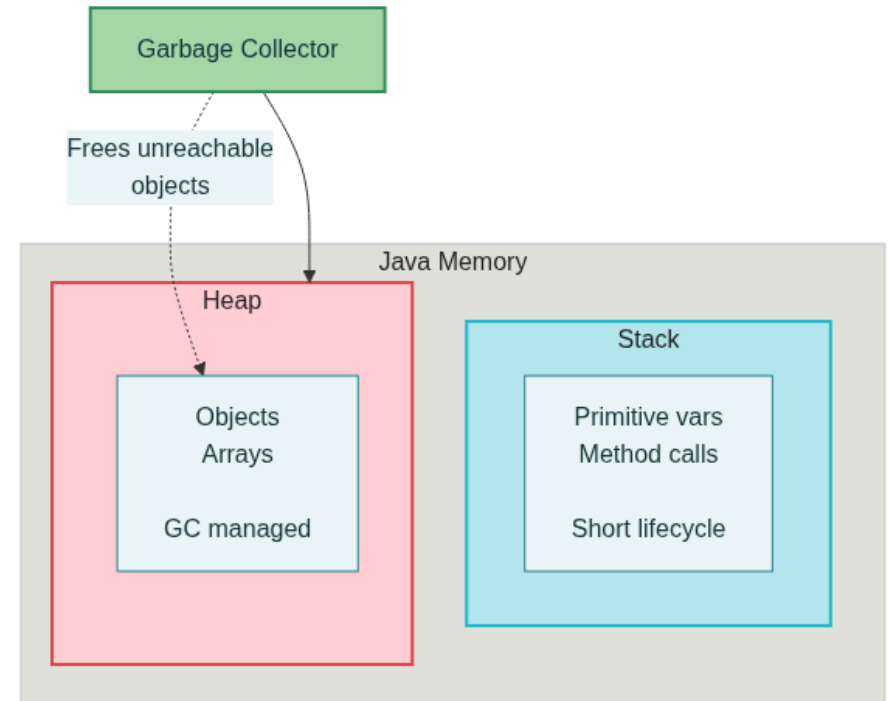
Si no se pone nada se puede acceder libremente al atributo cuando se declare un objeto, pero siempre desde el mismo paquete (esto se verá más adelante).

El tipo de acceso **protected** se explicará cuando se estudie el tema de la herencia

ALMACENAMIENTO EN MEMORIA Y EL USO DEL GARBAGE COLLECTOR PARA LA LIBERACIÓN DE MEMORIA

¿Cómo gestiona Java la memoria?

- Java utiliza una zona de memoria llamada **Heap** donde se almacenan los objetos creados durante la ejecución del programa.
- Las variables primitivas (int, char, etc.) suelen almacenarse en la **pila (stack)**, mientras que los objetos y arrays están en el heap.
- Cuando una variable deja de usarse, el espacio de memoria no se libera automáticamente.
- **El papel del Garbage Collector**
- El **Garbage Collector (Recolector de Basura)** es un proceso automatizado de Java que identifica objetos que ya no tienen referencias activas y libera su memoria.
- Esto evita fugas de memoria y facilita la gestión automática, permitiendo a los desarrolladores centrarse en la lógica del programa.
- No es necesario liberar memoria manualmente (como en C/C++), Java lo hace automáticamente cuando determina que un objeto es inalcanzable.



LIMPIEZA O LIBERACIÓN DE MEMORIA

La memoria **stack** **sí se limpia automáticamente**, pero de una forma diferente al heap:

- En el **stack** se almacenan variables locales y llamadas a métodos con un ciclo de vida muy corto. La memoria se reserva y libera de forma automática y ordenada cuando un método se invoca y termina.
- Cuando un método termina, todas las variables locales (que están en el stack) se eliminan inmediatamente porque el stack "retrocede" al estado anterior. No necesita un recolector como el heap, es un proceso rápido de asignación y liberación en orden LIFO (Last In, First Out).

Por el contrario, en el **heap** la memoria asignada a objetos debe liberarse cuando los objetos ya no son referenciados. Aquí es donde entra el **Garbage Collector** para buscar y limpiar objetos que ya no se usan, de forma automática para evitar fugas de memoria.

En resumen:

- **Stack:** la limpieza es automática y ocurre cada vez que termina un método, eliminando todas sus variables locales.
- **Heap:** la limpieza es automática, pero se hace por el Garbage Collector que detecta qué objetos ya no tienen referencias y libera su memoria en momentos indeterminados.

